

Data Modelling

Animal Extinction in Australia

We define the scope of the project as the creation of a database that would organize information about and connecting to animal extinction in Australia such that the users would be able to infer causal links between the entities. Thus, the purpose of the database is to aid stakeholders with taking appropriate action in animal protection.

In our database we have modelled the following entities:

1. Animal

This class represents individual animals – both endangered and not (this fact is modelled by Animal Endangerment entity) with their characteristics.

2. Animal Endangerment

3. Animal Population

Animal Population allows us to model the change in numbers of the animal.

4. Location

Location is a key entity in the database as it enables the users to link potential causes to the endangerment.

5. Location of an Animal

This bridge table links individual animals to its location(s).

6. Natural Disaster

This entity was chosen to model potential negative consequences of natural disasters on animal species.

7. Climate

Climate entity provides additional information to the location entity but was modelled separately to allow to track the change in location attributes in time.

8. Urban Development

9. Disease Outbreaks

10. Diseases

11. Predator and Prey

Entities 8.-11. were chosen as potential negative influences on the animal endangerment.

Normalization

When designing our database, we directly used normalization principles to make sure the data was organized properly. Thus, instead of creating tables arbitrarily and fixing problems afterward, we thought about how the data was related in such a way to have it optimized.

Firstly, we made sure that the values in our tables are atomic in order to adhere to the 1NF, meaning that each field contains only one value rather than several values combined. For example, instead of having a location attribute in the Animal entity, where we would have to possibly store multiple locations given the fact that an animal may have multiple sites for its habitat, we have created a bridge table Location of an Animal. Here, we link individual animals to individual locations, thus all attributes are atomic and consequently, we avoid a direct many-to-many relationship between two entities.

1NF – Before:

Animal (violation: locations holds a list of values)

animal_id	scientific_name	common_name	locations (violation)
1	Phascolarctos cinereus	Koala	Daintree Rainforest, Snowy Mountains
2	Ornithorhynchus anatinus	Platypus	Snowy Mountains, Tasmanian Wilderness
3	Chelonia mydas	Green Sea Turtle	Great Barrier Reef

1NF – After:

Animal

animal_id	scientific_name	common_name
1	Phascolarctos cinereus	Koala
2	Ornithorhynchus anatinus	Platypus
3	Chelonia mydas	Green Sea Turtle

Location_Of_Animal (new bridge table)

animal_id	location_id
1	1 (Daintree Rainforest)
1	2 (Snowy Mountains)
2	2 (Snowy Mountains)
2	3 (Tasmanian Wilderness)
3	4 (Great Barrier Reef)

Similarly, Climate was modelled as a separate entity to Location as including climate as a Location attribute would require us to store multiple values for locations with multiple climates, which would violate the 1NF. Same applies to Urban Development – in one location multiple development projects can take place, thus we would be again required to store a list.

Secondly, to ensure our database was in 2NF, we made sure that the non-key attributes are dependent on the entire primary key of the table. For instance, the Population of Animal entity has a composite primary key which consists of animal_id, location_id, and population_year. The non-key attribute population_size is directly dependent on this combination since the population may differ for a particular animal, location, and year. We did not include any further information about the animal or the location as such attributes would not be related to the entire primary key and thus, they are kept in separate tables. Namely, the biome, area_sq_km, and protection_level describe the location itself, hence, they are stored in the Location table instead of being repeated in other tables.

2NF – Before:

Population_of_Animal (violation: biome and protection_level depend only on location_id)

animal_id	location_id	population_year	population_size	biome (violation)	protection_level (violation)
1	1	2010	15000	Tropical Forest	II
1	1	2020	9800	Tropical Forest	II
5	6	2020	25000	Temperate Forest	I

2NF – After:

Population_of_Animal

animal_id	location_id	population_year	population_size
1	1	2010	15000
1	1	2020	9800
5	6	2020	25000

Location

location_id	location_name	biome	protection_level
1	Daintree Rainforest	Tropical Forest	II
6	Tasmanian Wilderness	Temperate Forest	I

We took a similar approach for Endangerment of an Animal. This table has a composite primary key of animal_id and effect_year and the level attribute records the animal's endangerment category for the given year. This way we can monitor changes in the threat status of an animal over time, without having to repeat the other information about the animal. If we were to include animal information, this would again violate the 2NF as such information would not be dependent on the entire primary key.

Thirdly, 3NF requires that non-key attributes do not depend on other non-key attributes. We applied this idea to the Disease and Disease Outbreaks tables. Disease Outbreaks entity has a foreign key disease_id to connect each outbreak to the correct disease. If we were to include the attribute disease_type in this entity as well, it would violate the 3NF as disease_type would be dependent on a non-primary key disease_name instead of being dependent on the primary key outbreak_id, thus, we have created a separate table where we link a disease_id to its disease_name and disease_type.

3NF – Before:

Disease_Outbreaks (violation: disease_type depends on disease_name, not on outbreak_id)

outbreak_id	disease_name	disease_type (violation)	animal_id	location_id	outbreak_year	disease_mortality_percent
1	Devil Facial Tumour Disease	Cancer	5	6	1996	70.50
2	Devil Facial Tumour Disease	Cancer	5	6	2010	85.00
3	Chlamydia	Bacterial	1	1	2015	30.25

3NF – After:

Diseases

disease_id	disease_name	disease_type
1	Devil Facial Tumour Disease	Cancer
2	Chlamydia	Bacterial

Disease_Outbreaks

outbreak_id	disease_id	animal_id	location_id	outbreak_year	disease_mortality_percent
1	1	5	6	1996	70.50
2	1	5	6	2010	85.00
3	2	1	1	2015	30.25

Overall, normalization helped us organize our ERD so that each table has a clear purpose. It reduces duplicate data and lowers the chance of inconsistencies when information needs to be updated. Instead of repeating the same information in multiple tables, we use primary keys and foreign keys to connect related data. This makes the database easier to maintain, understand, and query.